Do You Need a Neural Network? Choosing a Fast Stand-In for Slow Physics Simulations

A practical comparison of Gaussian-process and neural-network stand-in models on 36 simulated engineering problems

Authors: [AUTHOR NAMES AND AFFILIATIONS — to be supplied by the authors]. **Corresponding author:** [to be supplied].

Abstract

Engineers often need to run the same slow physics simulation hundreds of times, for design, calibration or uncertainty studies. A common shortcut is to run it a limited number of times and train a fast stand-in model, called a surrogate, on the results. This paper asks a practical question: with a budget of 32 to 1,024 simulation runs, should you build the stand-in from a Gaussian process (GP) working on a reduced basis, or train a neural network? We compared five such models on 36 simulated problems modelled on six types of application (flow through rock, heat conduction, pollutant transport, thermal runaway, spreading fronts, and waves). On smooth problems with few inputs, the reduced-basis GP was far more accurate, sometimes by two or three orders of magnitude, and the reduced-basis family reached 0.1% error on 19 of the 36 problems within the budget, against 10 for the networks. For rough accuracy targets (10% or 1%) the two families were about equally good. Several of the neural networks' apparent wins came from the limits of the reduced basis rather than of the GP, and disappeared when the GP was applied to the full field. The GP's edge was not caused by the reduced basis; it came mainly from searching its hyper-parameters over a wide range, since kernel ridge with a wide search came within a factor of about 3 of it. Results shifted noticeably when the networks were trained longer, so we give a short checklist for fair comparisons. A quick pilot check meant to predict the winner from 32 runs did not work reliably. All problems are simulated and the networks were only lightly tuned.

Keywords: surrogate model; Gaussian process; neural network; reduced-order model; parametric PDE; simulation budget

1 Introduction

Many engineering decisions depend on a simulation that is too slow to run as often as we would like. Picking the best shape of a part, estimating how uncertain an input leaves the answer, or fitting a model to measurements can each take thousands of runs. The usual remedy is a *surrogate* (or stand-in) model: run the full simulation a limited number of times, say a few hundred, train a cheap model on the inputs and outputs, and then use the cheap model for everything else.

The practical question is which kind of model to build. One long-standing route compresses the simulation output with proper orthogonal decomposition (POD), which finds a small set of typical shapes, and then learns how the weights of those shapes depend on the inputs, often with a Gaussian process [Hesthaven & Ubbiali 2018; Guo & Hesthaven 2018; Bhattacharya et al. 2021]. The other route trains a neural network to produce the whole output directly, sometimes through a narrow internal layer [Fresca & Manzoni 2022; Franco et al. 2023]. Neural networks get most of the attention, but there are good reasons not to take their advantage for granted. Surveys of machine-learning solvers for fluid problems found that most papers claiming to beat a standard method had used a weak baseline [McGreivy & Hakim 2024]. Kernel and Gaussian-process methods have been reported to be competitive with DeepONet [Lu et al. 2021] and the Fourier neural operator [Li et al. 2021] [Batlle et al. 2024], and in a POD framework for extruder flows a Gaussian process was reported to have several advantages over a neural network [Bērziņš et al. 2020, preprint]. A recent comparison of neural and polynomial surrogates found no universally superior method [Westermann et al. 2026, preprint]. Whether a small POD basis can work at all depends on how well the set of solutions can be approximated by a few shapes (the decay of the Kolmogorov n-width) [Cohen & DeVore 2015], and this decay has been shown to be slow for wave problems with discontinuous data [Greif & Urban 2019]. Finally, how much tuning a method gets and how results are reported can decide a comparison, which is why reporting results as a function of the tuning budget has been advocated [Dodge et al. 2019; Bouthillier et al. 2021].

What a practitioner still lacks is a quantitative picture. How big is the difference, on which kinds of problems, at which budget? Why does one side win? Is there a cheap way to tell in advance? And how much do the comparison's own settings, such as how long the network is trained, change the answer? We tried to answer these questions with a controlled comparison on 36 simulated problems modelled on six types of application. We compared five stand-in models under the same tuning procedure, measured how many simulation runs each needs to reach a given accuracy, and ran extra experiments to find out *why* the winners win.

In brief, the Gaussian-process stand-ins were far more accurate than lightly tuned neural networks on smooth problems with a few inputs, and neither side had a consistent advantage on the rest. A good part of the networks' apparent wins on harder problems came from the limits of the reduced basis rather than from the GP, and the GP's advantage came mostly from how widely and carefully its settings were searched. The outcome also moved when we trained the networks longer. We built the paper around a practical checklist (Section 6) that follows from these findings.

2 What we compared

The two kinds of stand-in. In the *reduced-basis group* the simulation output is compressed with POD and a regressor predicts the weights of the typical shapes from the inputs. We used three regressors: a Gaussian process with a separate length-scale for each input (POD+GP), a polynomial fitted with a ridge penalty (POD+Poly), and a small neural network (POD+MLP). In the *direct-network group* a neural network produces the whole output at once, either as a plain multilayer perceptron or through a narrow bottleneck layer. We say "better" for a group when its best member, picked on held-out validation data and never on test data, has the smaller error.

Fair tuning. Every method got the same number of tuning trials: eight random draws of its settings (the first draw is a fixed default), scored on a random fifth of the training runs; the winner was then refitted on all runs. Networks were trained with Adam for 1,500 epochs in single precision. At the largest budget of 1,024 runs the training used mini-batches of 512 instead of the whole set, so the networks took twice as many optimisation steps there, which we keep in mind when reading the 1,024-run results.

Scoring. The error is the average relative L_2 error over 1,000 held-out inputs. To compare groups we use the $\log_{10}$ of the error ratio, network over reduced-basis: a value of 1 means the network's error is ten times larger, 2 means a hundred times, and negative values mean the network is better. Differences below 0.1 (about 25%) count as a tie. Each result is averaged over three random seeds that change the training subset and the tuning draws.

Why-experiments. To find out why one side wins we added more models: kernel ridge regression with a narrow or wide range of settings, a Gaussian process with a single length-scale, and a Gaussian process applied directly to the full output without POD. We also retrained the networks for longer, in double precision, and with a second-order polishing step (L-BFGS), to see how much of the networks' error was due to limited training. The technical details, all statistical tests and the full record of what we had decided in advance are in the Supplement (S1).

How we kept the analysis honest. The main hypotheses, code and success criteria were committed to version control before each group of runs. Several later experiments were designed after seeing earlier results, and one of them (the wide-range kernel ridge) was added because an independent AI-agent review of our own draft showed that an earlier control was too narrow. The Supplement lists which results came from which stage. The version-control history is our own, not an independent registry.

3 The test problems

Real simulation datasets with known accuracy are hard to share and verify, so we used our own simulators, each checked against exact or manufactured solutions or by grid refinement. The 36 problems fall into six application types (Table 1). **They are stand-ins modelled on these applications, not field or industrial data**; the point is to cover the range of behaviour (smooth or sharp, few or many inputs, steady or moving features) that practitioners meet. Each application type has two or more problems, often a gentle and a harder version.

Application type	What is simulated	Reduced basis better / tie / network better	Median $\log_{10}$ error ratio	Typical error: reduced basis / network
Flow through porous rock (9 problems, 3–32 inputs)	steady flow with an uncertain permeability field, one with a moving inclusion (a lens of different rock)	3 / 1 / 5	-0.13	4e-03 / 3e-03
Heat conduction (9 problems, 3–8 inputs)	heating by a moving source, uncertain conductivity or diffusivity, nonlinear conduction	7 / 0 / 2	2.58	2e-06 / 1e-03
Pollutant or heat carried by a flow (4 problems, 4 inputs)	advection–diffusion and pure transport of pulses	2 / 1 / 1	0.07	7e-03 / 7e-03
Thermal runaway (combustion-type) (2 problems, 3 inputs)	a nonlinear elliptic problem with a steep response near a critical parameter value	2 / 0 / 0	3.19	5e-07 / 7e-04
Spreading fronts and reactions (4 problems, 3–4 inputs)	travelling reaction–diffusion fronts and nonlinear reactions	3 / 0 / 1	1.45	2e-04 / 9e-04
Waves, shocks and acoustics (8 problems, 3–4 inputs)	viscous shocks, a 1-D wave, solitons, Helmholtz-type oscillations	6 / 1 / 1	0.53	1e-03 / 5e-03

Table 1. The 36 test problems by application type. The last two columns are medians over the problems of each type at 1,024 training runs: the $\log_{10}$ error ratio (positive: the reduced-basis group is better) and the typical test error of each group.

For every problem we generated 4,096 candidate training inputs and 1,000 held-out test inputs, plus 500 inputs outside the training range to see what happens when a model has to extrapolate. We looked at 32 to 1,024 training runs for 24 problems and at 64, 256 and 1,024 runs for the other 12.

4 Results

4.1 Who wins, and by how much

The answer depends heavily on the budget and the problem type (Figure 1, Table 1). With very few runs (32) the two groups are mostly tied: of the 24 main problems, 5 favour the reduced-basis group, 5 the networks, and 14 are ties. With 1,024 runs the reduced-basis group is clearly better on 13 problems, the networks on 8, and 3 are ties (the number of network wins is 6 if the tie band is widened to 0.15 and 10 if it is narrowed to 0.05). Earlier we had read the small-budget results as "networks win most cases", but most of those were ties.

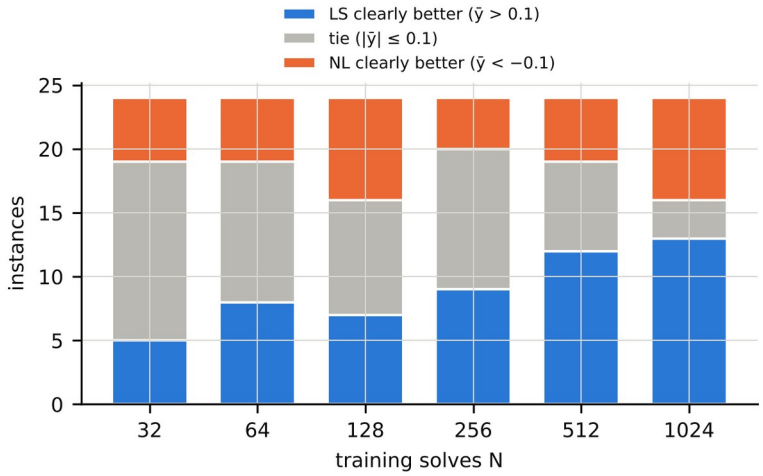


Figure 1. *For each training budget, on how many of the 24 main test problems the reduced-basis models were clearly more accurate than the best neural network (blue), the two were within about 25% of each other (grey), or the neural network was clearly better (orange).*

When the reduced basis wins, it can win by a lot: with 1,024 runs the networks' error is about 2,000 times larger on the thermal-runaway problem (bratu_L1), 530 times on a nonlinear heat problem (nlheat_L1), and 180 times on a viscous shock (burgers_L1). The largest margin in the other direction is a factor of 2.8. The eight extra smooth problems we added later (reaction–diffusion, Helmholtz, heat with uncertain diffusivity) all went the same way, with factors from about 10 to 850; six of the eight have only three inputs, so this mainly confirms that smooth low-dimensional problems favour the GP.

These large factors come with caveats. They refer to the interpolation test set, to 1,024 runs, and to networks trained for 1,500 epochs in single precision. On the extrapolation test set the gaps are smaller (for example 3.3 → 1.5 on the log scale for bratu_L1, and 2.7 → 1.8 for nlheat_L1); with longer and more careful network training they shrink by roughly half a decade (Section 4.5). A fair summary is "up to about three orders of magnitude in the most favourable setting, one to two orders of magnitude after stronger network training".

By application type, reduced-basis models were ahead on most heat-conduction and reaction problems, on the thermal-runaway problems and on the simple porous-flow problems. The networks led on a few problems with sharp moving features (transport_L2, kdv_L1) and on the porous-flow problems with a moving inclusion or many inputs (incl_L1, incl_L2, darcy_L2, darcyhd_L1); as Section 4.3 shows, a full-field GP often reverses the inclusion cases. For the problems with 16 and 32 inputs the results are not consistent across seeds and we cannot say which family is better there.

4.2 How many runs do you need?

Often the real question is how many simulations are needed to reach a given accuracy. Table 2 counts the problems on which each group's best model reached a target error within the 1,024 runs of our study. For a coarse target of 10% both groups got there on all 36 problems, usually with 64 runs or fewer. For 1%, they were about equal (27 against 28 problems). The difference shows up at the demanding end: 19 problems reach 0.1% error with the reduced-basis group, against 10 with the networks. The extra accuracy is mostly on smooth, low-dimensional problems; on several others (flows with moving inclusions, wave problems, shocks with sharp fronts) neither group reaches 1% within the budget.

Target accuracy (mean relative error)	Problems where the reduced-basis group gets there within 1024 runs	Problems where the neural group gets there within 1024 runs
10% error	36	36
1% error	27	28
0.1% error	19	10

Table 2. Number of the 36 problems on which the best model of each group reached the target mean error within the study's training budget (at most 1,024 runs; the budget grid is coarser for 12 problems).

Training time is a minor cost compared with the simulations themselves but is not negligible. At 1,024 runs, tuning and fitting took a median of about 44 seconds of CPU time for the GP model, 86 for the POD+network, 116–139 for the direct networks, and a third of a second for the polynomial.

4.3 Why the Gaussian process does so well

We tested several explanations (Figure 2).

It is not the reduced basis. A Gaussian process applied directly to the full output field, without any POD, was as accurate as the reduced-basis GP where the latter did well (median error ratio 0.95). Where networks had beaten the POD-based GP, the full-field GP often beat the networks again. On the heat problem with a narrow moving source (heat2d_L2) and the two inclusion problems (incl_L1, incl_L2), the error of the POD+GP model is the same as the error of the POD projection with 32 modes (0.055, 0.0020 and 0.0046 against 0.054, 0.0020 and 0.0042), so POD itself is the bottleneck and the networks, which are not confined to a basis, get below it. When we ran the full-field GP on the two moving-feature problems added later, it beat the best network by 1.2 and 1.0 orders of magnitude at 1,024 runs, where the POD+GP had lost to the networks.

It is largely how the settings are searched. Kernel ridge regression is the same kind of model without the Gaussian process's automatic fitting of its settings. With the narrow range of settings we first gave it, it was a median 14 times less accurate than the GP in the cells where the GP clearly wins. After an independent review of our draft (by an AI agent) pointed out that this range could not reach the very small regularisation that near-perfect fits need, we repeated the test with a wide range. The gap shrank to a factor of 6.5 with the same eight tuning trials and to 2.7 with 32 trials, and the wide-range kernel ridge beat the best neural network clearly in 64% of those cells. A polynomial, which has no kernel and no automatic fitting at all, was a median 10 times worse than the GP but still clearly beat the best network in 23 of the 52 cells where the GP did. Taken together, the advantage belongs to smooth regressors with a carefully and widely tuned regularisation, and not to the Gaussian process as such. We did not isolate the contribution of the GP's marginal-likelihood fitting, because its cost is not matched by our kernel-ridge search.

It is not a matter of the direct network being worse than the POD network. A direct network beat the network on POD coefficients by more than 25% in 17 of the 24 problems at 1,024 runs, so putting POD in front of a network did not help it.

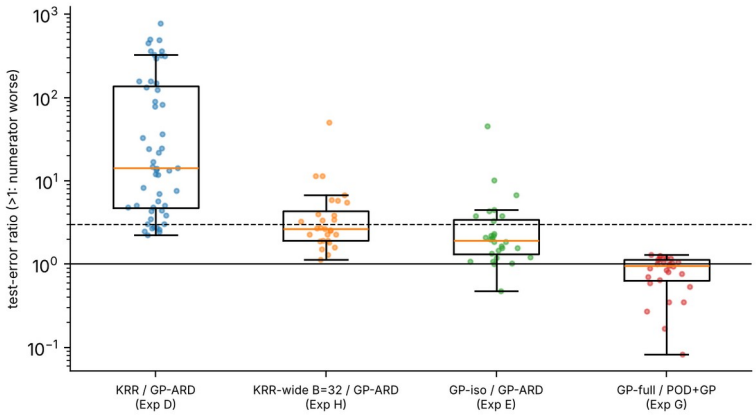


Figure 2. *How much worse each alternative is than the Gaussian-process reduced model, on problems where that model clearly beats the best neural network. Each dot is one problem and budget; the dashed line marks a factor of 3. From the left: kernel ridge with a narrow search range, kernel ridge with a wide search range, a Gaussian process with one length-scale, and a Gaussian process on the full field without a reduced basis.*

4.4 Can a quick pilot check tell you which to use?

It would be convenient to run 32 simulations, look at how quickly POD compresses them, and decide. We froze such a rule from the original data and tested it on eight new problems. It did not work convincingly: the correlation was in the expected direction but not significant (ρ = -0.38, p = 0.180) and the rule's gain over a baseline was borderline (p = 0.051 after correction). Most of that borderline gain came from two nonlinear-heat problems, and without them the correlation changes sign. The pilot statistic also tracks the number of inputs: it is strongly negative only on three-input problems. Follow-up experiments agreed with a simple reading: smooth problems with few inputs favour the GP by a lot, and a pilot statistic mostly detects that. We would not rely on it beyond an informal check of smoothness and dimension. Full details are in the Supplement.

4.5 How much do the comparison's own settings matter?

Three settings changed the answer noticeably.

Training the networks longer. Retraining the stored best networks for four times as many epochs improved nine in ten of them and moved the comparison toward the networks in all 15 of the 48 (problem, budget) cases whose classification changed; 9 ties became network wins. Double precision and a second-order polish lowered the networks' error by a further half decade on average. The largest gaps survived: on 5 of the 7 problems where the reduced basis led by a factor of 10 or more, it still did after the stronger training, and on the other two (darcy_L1 and heat2d_L1) it led by factors of about 4 and 9. The networks' error also stayed around $2{\cdot}10^{-4}$ at best, far above the best GP errors of 10^{-7} to 10^{-5}, which suggests training limits rather than a lack of capacity; but we did not explore every way of tuning a network, and a thoroughly tuned network could do better.

Unequal tuning. When the networks got eight tuning trials and the reduced-basis models only one, the winner switched to the networks in 10 problem-budget cases and in the other direction in 1. Giving one side more tuning moves the comparison toward that side, as one would expect; this is why we gave every method the same number of trials. Equal numbers of trials do not mean equal effort, though: the GP's search has only eight possible settings but then fits its length-scales continuously, while the network search is continuous in several dimensions.

Random draws. Repeating the comparison with different halves of the random tuning draws gave the same verdict (reduced-basis better, tie, or network better) in 83% of cases; the sign of a clear difference agreed in 99%. Ties and small differences should therefore not be over-interpreted, while large gaps are stable. Among cells where the networks were clearly better, only 57% were clearly better in all three seeds, against 80% for the reduced basis.

5 Worked examples

The following three examples show what the numbers mean for a practitioner. All values are for our simulated stand-ins.

Uncertain permeability in a porous medium (darcy, darcyhd, darcylin). The simple version has 4 inputs. With 256 runs the reduced-basis group reaches a mean error of 3.2e-04 and the best network 4.2e-03; with 1,024 runs the

figures are 1.5e-04 and 2.1e-03. If 1% error is enough, either approach works from about 64 runs. With 16 inputs (darcyhd_L1) both groups are stuck at an error of about 6e-02 even with 1,024 runs, and they are within about 30% of each other; here the difficulty comes from the number of inputs, not from the choice of model.

A part heated by a moving source (heat2d, heatmv3). With a broad source (heat2d_L1) the best reduced-basis model needs about 64 runs to reach 1% error and 256 for 0.1%; the direct networks need about 128 for 1% and never reach 0.1% in our study. With a narrow moving source (heat2d_L2) neither reaches 1% error by 1,024 runs, and the networks are ahead of the POD-based GP by a small margin. But the error of the POD model equals the projection error of its 32 modes, so a full-field GP is a better choice there. This is the type of problem where the form of the stand-in matters more than the regressor on top of it.

Thermal runaway (bratu). Near a critical parameter the response of this problem changes steeply, yet it depends smoothly on its three inputs. The reduced-basis group reaches 0.1% error with only 32 runs on both versions (bratu_L1 and bratu_L2); the best network needs 1,024 runs for the same accuracy. At 256 runs the error is 1e-06 against 1e-03. If your real problem has few inputs and a smooth response, try a Gaussian process first, and you may need far fewer simulations than you expect.

6 A practical checklist

We draw the following advice from the experiments. The strength of the evidence is noted in each case.

1. **Always run a well-tuned Gaussian-process or kernel baseline before you train a neural network.** On smooth problems with up to about four inputs it was usually the most accurate option, and it needs no long training. Give its regularisation a wide search range. (Strong for $p \le 4$ on our problems; unresolved for $p \ge 10$.)
2. **Add a polynomial baseline.** It takes a fraction of a second and, on our problems, still clearly beat the best neural network in nearly half the cells where the GP did. (Moderate.)
3. **Check whether POD is your bottleneck.** If the error of your stand-in equals the error of projecting the test data onto the same number of modes, a larger basis or a model without a basis will help more than a better regressor. (Strong; seen on three problems.)
4. **Decide your accuracy target first.** For 10% or even 1% error the two families are about equally good, and a network may be the more convenient choice. For 0.1% or better, the reduced-basis route reached the target on about twice as many problems (19 against 10 of 36). (Moderate; based on counts over 36 problems.)
5. **If you use a neural network, report and vary its training budget.** Training longer, in double precision and with a polishing step, improved most networks and changed the outcome on about a third of our cases. Do not conclude that a network cannot do better from a single 1,500-epoch run. (Strong.)
6. **Give the competing methods the same tuning effort and say what that effort was.** Unequal tuning moved the winner in the direction of the more heavily tuned side. (Moderate.)
7. **Report ties as ties.** Differences below about 25% flipped with the random draw of tuning settings in a sizeable minority of cases, so do not declare a winner on them. (Moderate.)
8. **Do not rely on a pilot statistic to choose.** The quick check we tried was not reliable. (Weak evidence against, inconclusive.)

7 What could go wrong

All our problems are simulated stand-ins, solved by our own code, on modest grids and with 3 to 32 inputs. We did not use field or industrial data, so the conclusions say how methods compare on these kinds of problems, not how they will perform on a particular application. The neural networks were only lightly tuned (eight random draws, one training recipe, shared draws across problems), and the 1,024-run results are affected by the doubling of the training steps. Better-tuned networks, other architectures such as Fourier neural operators, or very large budgets could change the picture; the evidence here favours the GP route for smooth low-dimensional problems and says little about problems with many inputs. Most of our statistical evidence is descriptive: we have 24 main problems in 12 families, the cases within a family are not independent, and several of the follow-up experiments were designed after we had seen earlier results. Parts of the supporting analysis, including the audits, were done with the help of AI agents rather than human reviewers.

8 Conclusion

For slow simulations of smooth problems with a handful of inputs, a Gaussian-process stand-in is a strong default: it was usually far more accurate than lightly tuned neural networks, needed fewer simulation runs for high accuracy, and was cheap to build. Its advantage is not about the reduced basis; it is mainly about a smooth regressor with carefully searched regularisation. Neural networks were competitive for modest accuracy targets and for some problems with sharp moving features, and on harder or higher-dimensional problems we could not name a clear winner. The outcome of a comparison depends on the training budget, the tuning effort and the random draws, so a fair comparison needs equal effort, tie-aware reporting and a well-tuned Gaussian-process baseline.

Declarations

Author contributions, affiliations, funding and competing interests: [to be supplied by the authors]. *Use of AI tools:* the study design, code, analyses, audits and the drafting of this paper were carried out with an AI system (Claude) working as a research assistant under the authors' direction; the audit and review rounds were performed by AI agents, not by human reviewers. The authors must verify every number, citation and claim and take responsibility for the text. *Data and code availability:* all data are generated by the code in the repository (`Reinforced_Research_Project/code`) from instance names and seeds; raw outputs are under `results/`; a permanent archive such as a Zenodo DOI has not yet been created. *Supplement S1:* technical report with all hypotheses, tests, the experiment-by-experiment verdicts and the deviation log.

References

[1] Batlle P, Darcy M, Hosseini B, Owhadi H (2024) Kernel methods are competitive for operator learning. J Comput Phys 496:112549. 10.1016/j.jcp.2023.112549 / arXiv:2304.13202

[2] Bhattacharya K, Hosseini B, Kovachki NB, Stuart AM (2021) Model reduction and neural networks for parametric PDEs. SMAI J Comput Math 7:121-157. 10.5802/smai-jcm.74

[3] Bouthillier X, Delaunay P, Bronzi M, et al. (2021) Accounting for variance in machine learning benchmarks. MLSys 2021. arXiv:2103.03098

[4] Bērziņš A, Helmig J, Key F, Elgeti S (2020) Standardized non-intrusive reduced order modeling using different regression models with application to complex flow problems. arXiv:2006.13706.

[5] Cohen A, DeVore R (2015) Approximation of high-dimensional parametric PDEs. Acta Numerica 24:1-159. 10.1017/s0962492915000033

[6] Dodge J, Gururangan S, Card D, Schwartz R, Smith NA (2019) Show your work: improved reporting of experimental results. EMNLP-IJCNLP 2019:2185-2194. 10.18653/v1/d19-1224

[7] Franco NR, Manzoni A, Zunino P (2023) A deep learning approach to reduced order modelling of parameter dependent partial differential equations. Math Comp 92(340):483-524. 10.1090/mcom/3781

[8] Fresca S, Manzoni A (2022) POD-DL-ROM: enhancing deep learning-based reduced order models for nonlinear parametrized PDEs by proper orthogonal decomposition. Comput Methods Appl Mech Eng 388:114181. 10.1016/j.cma.2021.114181

[9] Greif C, Urban K (2019) Decay of the Kolmogorov N-width for wave problems. Appl Math Lett 96:216-222. 10.1016/j.aml.2019.05.013

[10] Guo M, Hesthaven JS (2018) Reduced order modeling for nonlinear structural analysis using Gaussian process regression. Comput Methods Appl Mech Eng 341:807-826. 10.1016/j.cma.2018.07.017

[11] Hesthaven JS, Ubbiali S (2018) Non-intrusive reduced order modeling of nonlinear problems using neural networks. J Comput Phys 363:55-78. 10.1016/j.jcp.2018.02.037

[12] Li Z, Kovachki N, Azizzadenesheli K, et al. (2021) Fourier neural operator for parametric partial differential equations. ICLR 2021. arXiv:2010.08895

[13] Lu L, Jin P, Pang G, Zhang Z, Karniadakis GE (2021) Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. Nat Mach Intell 3:218-229. 10.1038/s42256-021-00302-5

[14] McGreivy N, Hakim A (2024) Weak baselines and reporting biases lead to overoptimism in machine learning for fluid-related partial differential equations. Nat Mach Intell 6(10):1256-1269. 10.1038/s42256-024-00897-5

[15] Westermann J, Huber B, O'Leary-Roseberry T (2026) Performance of neural and polynomial operator surrogates. arXiv:2604.00689.